

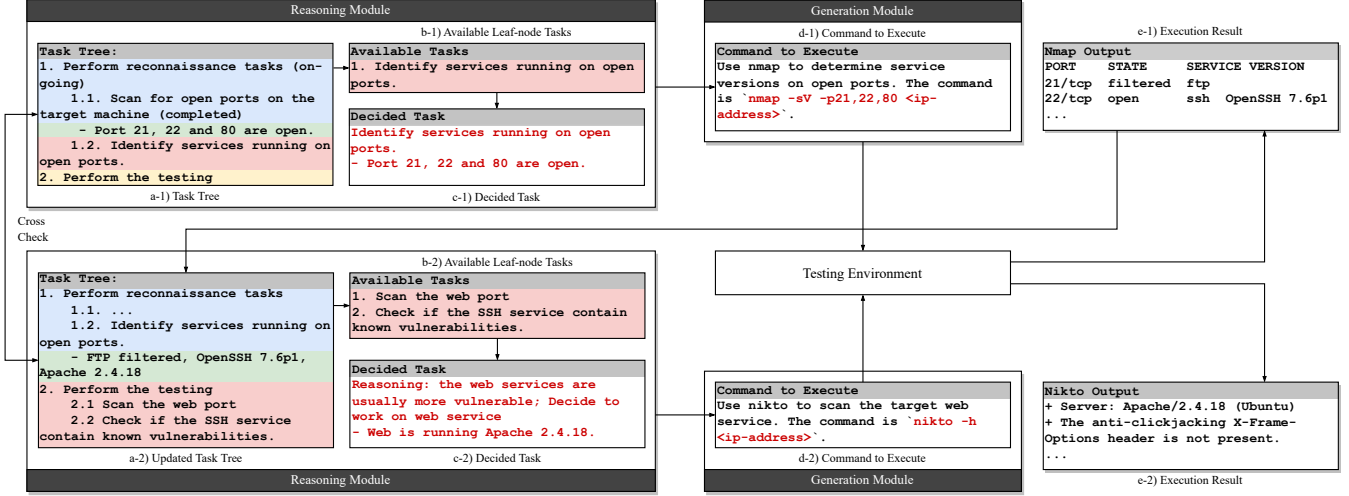


Figure 5: A demonstration of the task-tree update process on the testing target *HTB-Carrier*

cuted in the testing environment, and the execution result is conveyed to the Reasoning Module to update the PTT. In a-2), the Reasoning Module integrates the previous scanning result into the PTT, cross-referencing it with the earlier PTT to update only the leaf nodes. It then looks for the available tasks to execute. In this case, two tasks emerge: scanning the web service on port 80 and checking the SSH service for known vulnerabilities. The LLM evaluates which task is more promising and chooses to investigate the web service, often seen as more vulnerable. This task is passed to the Generation Module. The Generation Module turns this general task into a detailed process, employing *nikto* [50], a commonly used web scanning script. The iterative process continues until the tester completes the penetration testing task.

5.5 Parsing Module

The **Parsing Module** operates as a supportive interface, enabling effective processing of the natural language information exchanged between the user and the other two core modules. Two needs can primarily justify the existence of this module. First, security testing tool outputs are typically verbose, laden with extraneous details, making it computationally expensive and unnecessarily redundant to feed these extended outputs directly into the LLMs. Second, users without specialized knowledge in the security domain may struggle to extract key insights from security testing outputs, presenting challenges in summarizing crucial testing information. Consequently, the Parsing Module is essential in streamlining and condensing this information.

In PENTESTGPT, the Parsing Module is devised to handle four distinct types of information: (1) *user intentions*, which are directives provided by the user to dictate the next course of action, (2) *security testing tool outputs*, which represent the raw outputs generated by an array of security testing tools, (3)

raw HTTP web information, which encompasses all raw information derived from HTTP web interfaces, and (4) *source codes* extracted during the penetration testing process. Users must specify the category of the information they provide, and each category is paired with a set of carefully designed prompts. For source code analysis, we integrate the GPT-4 code interpreter [51] to execute the task.

5.6 Active Feedback

While LLMs can produce insightful outputs, their outcomes sometimes require revisions. To facilitate this, we introduce an interactive handle in PENTESTGPT, known as active feedback, which allows the user to interact directly with the Reasoning Module. A vital feature of this process is that it does not alter the context within the Reasoning Module unless the user explicitly desires to update some information. The reasoning context, including the PTT, is stored as a fixed chunk of tokens. This chunk of tokens is provided to a new LLM session during an active feedback interaction, and users can pose questions regarding them. This ensures that the original session remains unaffected, and users can always query the reasoning context without making unnecessary changes. If the user believes it necessary to update the PTT, they can explicitly instruct the model to update the reasoning context history accordingly. This provides a robust and flexible framework for the user to participate in the decision-making process actively.

5.7 Discussion

We explore various design alternatives for PENTESTGPT to tackle the challenges identified in Exploratory Study. We have experimented with different designs, and here we discuss some key decisions.

Addressing Context Loss with Token Size: a straightforward solution to alleviate context loss is the employment of LLM models with an extended token size. For instance, GPT-4 provides versions with 8k and 32k token size limits. This approach, however, confronts two substantial challenges. First, even a 32k token size might be inadequate for penetration testing scenarios, as the output of a single testing tool like *dirbuster* [52] may comprise thousands of tokens. Consequently, GPT-4 with a 32k limit cannot retain the entire testing context. Second, even when the entire conversation history fits within the 32k token boundary, the API may still skew towards recent content, focusing on local tasks and overlooking broader context. These issues guided us in formulating the design for the Reasoning Module and the Parsing Module.

Vector Database to Improve Context Length: Another technique to enhance the context length of LLMs involves a vector database [53, 54]. By transmuting data into vector embeddings, LLMs can efficiently store and retrieve information, practically creating long-term memory. Theoretically, penetration testing tool outputs could be archived in the vector database. In practice, though, we observe that many results closely resemble and vary in only nuanced ways. This similarity often leads to confused information retrieval. Solely relying on a vector database fails to overcome context loss in penetration testing tasks. Integrating the vector database into the design of PENTESTGPT is an avenue for future research.

Precision in Information Extraction: Precise information extraction is crucial for conserving token usage and avoiding verbosity in LLMs [55, 56]. Rule-based methods are commonly employed to extract diverse information. However, rule-based techniques are engineeringly expensive given natural language’s inherent complexity and the variety of information types in penetration testing. We devise the Parsing Module to manage several general input information types, a strategy found to be both feasible and efficient.

Limitations of LLMs: LLMs are not an all-encompassing solution. Present LLMs exhibit flaws, including hallucination [57, 58] and outdated knowledge. Our mitigation efforts, such as implementing task tree verification to ward off hallucination, might not completely prevent the Reasoning Module from producing erroneous outcomes. Thus, a human-in-the-loop strategy becomes vital, facilitating the input of necessary expertise and guidance to steer LLMs effectively.

6 Evaluation

In this section, we assess the performance of PENTESTGPT, focusing on the following four research questions:

RQ3 (Performance): How does the performance of PENTESTGPT compare with that of native LLM models and human experts?

RQ4 (Strategy): Does PENTESTGPT employ different problem-solving strategies compared to those utilized by LLMs or human experts?

RQ5 (Ablation): How does each module within PENTESTGPT contribute to the overall penetration testing performance?

RQ6 (Practicality): Is PENTESTGPT practical and effective in real-world penetration testing tasks?

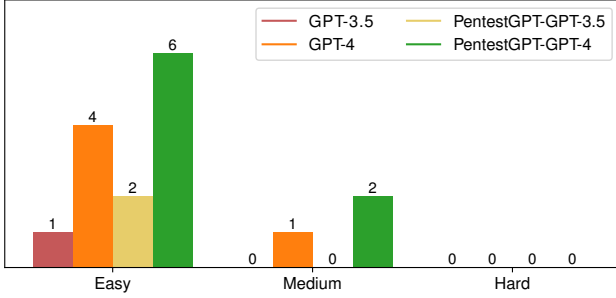
6.1 Evaluation Settings

We implement PENTESTGPT with 1,900 lines of Python3 code and 740 lines of prompts, available at our anonymized project website [32]. We evaluate its performance over the benchmark constructed in Section 3, and additional real-world penetration testing machines (Section 6.5). In this evaluation, we integrate PENTESTGPT with GPT-3.5 and GPT-4 to form two working versions: PENTESTGPT-GPT-3.5 and PENTESTGPT-GPT-4. Due to the lack of API access, we do not select other LLM models, such as Bard. In line with our previous experiments, we use the same experiment environment setting and instruct PENTESTGPT to only use the non-automated penetration testing tools.

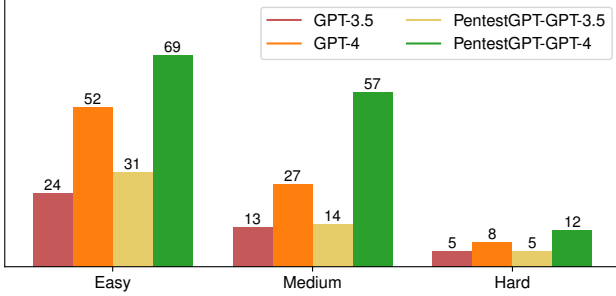
6.2 Performance Evaluation (RQ3)

The overall task completion status of PENTESTGPT-GPT-3.5, PENTESTGPT-GPT-4, and the naive usage of LLMs is illustrated in Figure 6a. As the Figure shows, our solutions powered by LLMs demonstrate superior penetration testing capabilities compared to the naive application of LLMs. Specifically, PENTESTGPT-GPT-4 surpasses the other three solutions, successfully solving 6 out of 7 easy difficulty targets and 2 out of 4 medium difficulty targets. This performance indicates that PENTESTGPT-GPT-4 can handle penetration testing targets ranging from easy to medium difficulty levels. Meanwhile, PENTESTGPT-GPT-3.5 manages to solve only two challenges of easy difficulty, a discrepancy that can be attributed to GPT-3.5 lacking the knowledge related to penetration testing found in GPT-4.

The sub-task completion status of PENTESTGPT-GPT-3.5, PENTESTGPT-GPT-4, and the naive usage of LLM is shown in Figure 6b. As the Figure illustrates, both PENTESTGPT-GPT-3.5 and PENTESTGPT-GPT-4 perform better than the standard utilization of LLMs. It is noteworthy that PENTESTGPT-GPT-4 not only solves one more medium difficulty target compared to naive GPT-4 but also accomplishes 111% more sub-tasks (57 vs. 27). This highlights that our design effectively addresses context loss challenges and leads to more promising testing results. Nevertheless, all the solutions struggle with hard difficulty testing targets. As elaborated in Section 4, hard difficulty targets typically demand a deep understanding from the penetration tester. To reach testing objectives, they may require modifications to existing penetration testing tools or scripts. Our design does not expand the LLMs’ knowledge of vulnerabilities, so it does not notably enhance performance on these more complex targets.



(a) Overall completion status.



(b) Subtask completion status.

Figure 6: The performance of GPT-3.5, GPT-4, PENTESTGPT-GPT-3.5, and PENTESTGPT-GPT-4 on overall target completion and sub-task completion.

6.3 Strategy Evaluation (RQ4)

We analyze PENTESTGPT’s problem-solving methods, comparing them with LLMs and human experts. Through manual examination, we identify PENTESTGPT’s approach to penetration testing. Notably, PENTESTGPT breaks down tasks similarly to human experts and prioritizes effectively. Rather than just addressing the latest identified task, PENTESTGPT identifies key sub-tasks that can result in success.

Figure 7 contrasts the strategies of GPT-4 and PENTESTGPT on the VulnHub machine, *Hackable II* [59]. This machine features two vulnerabilities: an FTP service for file uploads and a web service to view FTP files. A valid exploit requires both services. The figure shows GPT-4 starting with the FTP service and identifying the upload vulnerability (❶-❸). Yet, it does not link this to the web service, causing an incomplete exploit. In contrast, PENTESTGPT shifts between the FTP and web services. It first explores both services (❶-❷), then focuses on the FTP (❸-❹), realizing the FTP and web files are identical. With this insight, PENTESTGPT instructs the tester to upload a shell (❺), achieving a successful reverse shell (❻). This matches the solution guide and underscores PENTESTGPT’s adeptness at integrating various testing aspects.

Our second observation is that although PENTESTGPT behaves more similarly to human experts, it still exhibits some strategies that humans will not apply. For instance, PENTEST-

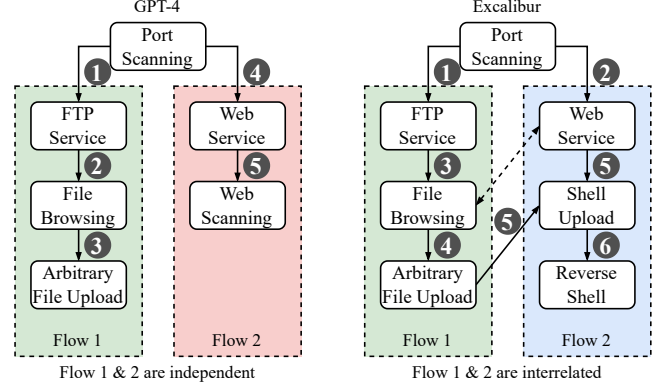


Figure 7: Penetration testing strategy comparison between GPT-3.5 and PENTESTGPT on *VulnHub-Hackable II*.

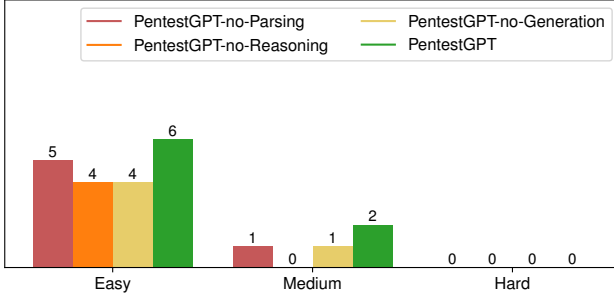
GPT still prioritizes brute-force attacks before vulnerability scanning. This is obvious in cases where PENTESTGPT always tries to brute-force the SSH service on target machines.

We analyze cases where penetration testing with PENTESTGPT failed, identifying three primary limitations. First, PENTESTGPT struggles with image interpretation. LLMs are unable to process images, which are crucial in certain penetration testing scenarios. Addressing this limitation may require the development of advanced multimodal models that can interpret both text and visual data. Second, PENTESTGPT lacks the ability to employ certain social engineering techniques and to detect subtle cues. For example, while a human tester might generate a brute-force wordlist from information extracted from a target service, PENTESTGPT can retrieve names from a web service but fails to guide the usage of tools needed to create a wordlist from these names. Third, the models struggle with accurate exploitation code construction within a limited number of trials. Despite some proficiency in code comprehension and generation, the LLM falls short in producing detailed exploitation scripts, particularly with low-level bytecode operations. These limitations underline the necessity for improvement in areas where human insight and intricate reasoning are still more proficient than automated solutions.

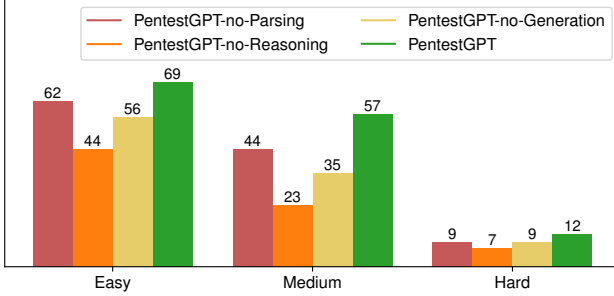
6.4 Ablation Study (RQ5)

We perform an ablation study on how the three modules: Reasoning Module, Generation Module, and Parsing Module, contribute to the performance of PENTESTGPT. We implement three variants:

1. PENTESTGPT-NO-PARSING: the Parsing Module is deactivated, causing all data to be directly fed into the system.
2. PENTESTGPT-NO-GENERATION: the Generation Module is deactivated, leading to the completion of task gen-



(a) Overall completion status



(b) Sub-task completion status

Figure 8: The performance of PENTESTGPT, PENTESTGPT-NO-ANNOTATION, PENTESTGPT-OPERATION-ONLY, and PENTESTGPT-PARAMETER-ONLY on both normalized average code coverage (μLOC) and bug detection.

eration within the Reasoning Module itself. The prompts for task generation remain consistent.

3. PENTESTGPT-NO-REASONING: the Reasoning Module is disabled. Instead of PTT, this variant adopts the same methodology utilized with LLMs for penetration testing, as delineated in the Exploratory Study.

All the variants are integrated with GPT-4 API for testing.

Figure 8 presents the outcomes of three tested variants on our benchmarks. Among these, PENTESTGPT consistently outperforms the ablation baselines in both target and sub-task completion. Our primary observations include: (1) Without its Parsing Module, PENTESTGPT-NO-PARSING sees only a slight drop in performance for task and sub-task completion. Though parsing aids in penetration testing, the 32k token limit generally covers diverse outputs. The Reasoning Module’s design, which retains the full testing context, compensates for the absence of the Parsing Module, ensuring minimal performance reduction. (2) PENTESTGPT-NO-REASONING has the lowest success, achieving just 53.6% of the sub-tasks of the full variant. This is even lower than the basic GPT-4 setup. The Generation Module’s added sub-tasks distort the LLM context. The mismatched prompts and extended generation output cloud the original context, causing the test’s failure. (3) PENTESTGPT-NO-GENERATION slightly surpasses the basic GPT-4. Without the Generation Module, the process

Table 5: PENTESTGPT performance over the active Hack-TheBox Challenges.

Machine	Difficulty	Completions	Completed Users	Cost (USD)
Sau	Easy	5/5 (✓)	4798	15.2
Pilgrimage	Easy	3/5 (✓)	5474	12.6
Topology	Easy	0/5 (✗)	4500	8.3
PC	Easy	4/5 (✓)	6061	16.1
MonitorsTwo	Easy	3/5 (✓)	8684	9.2
Authority	Medium	0/5 (✗)	1209	11.5
Sandworm	Medium	0/5 (✗)	2106	10.2
Jupiter	Medium	0/5 (✗)	1494	6.6
Agile	Medium	2/5 (✓)	4395	22.5
OnlyForYou	Medium	0/5 (✗)	2296	19.3
Total	-	17/50 (6)	-	131.5

Table 6: PENTESTGPT performance over picoMini CTF.

Challenge	Category	Score	Completions
<i>login</i>	web	100	5/5 (✓)
<i>advance-potion-making</i>	forensics	100	3/5 (✓)
<i>spelling-quiz</i>	crypto	100	4/5 (✓)
<i>caas</i>	web	150	2/5 (✓)
<i>XtrOrdinary</i>	crypto	150	5/5 (✓)
<i>triplesecure</i>	crypto	150	3/5 (✓)
<i>clutteroverflow</i>	binary	150	1/5 (✓)
<i>not crypto</i>	reverse	150	0/5 (✗)
<i>scrambled-bytes</i>	forensics	200	0/5 (✗)
<i>breadth</i>	reverse	200	0/5 (✗)
<i>notepad</i>	web	250	1/5 (✓)
<i>college-rowing-team</i>	crypto	250	2/5 (✓)
<i>fermat-strings</i>	binary	250	0/5 (✗)
<i>corrupt-key-1</i>	crypto	350	0/5 (✗)
<i>SaaS</i>	binary	350	0/5 (✗)
<i>risky business</i>	reverse	350	0/5 (✗)
<i>homework</i>	binary	400	0/5 (✗)
<i>lockdown-horses</i>	binary	450	0/5 (✗)
<i>corrupt-key-2</i>	crypto	500	0/5 (✗)
<i>vr-school</i>	binary	500	0/5 (✗)
<i>MATRIX</i>	reverse	500	0/5 (✗)

mirrors standard LLM usage. The module’s main role is guiding precise testing operations. Without it, testers might require additional information to use essential tools or scripts.

6.5 Practicality Study (RQ6)

We demonstrate PENTESTGPT’s applicability in real-world penetration testing scenarios, extending beyond standardized benchmarks. For this analysis, we deploy PENTESTGPT in two distinct challenge formats: (1) HackTheBox (HTB) active machine challenges, which present a series of real-world penetration testing scenarios accessible to a global audience. We selected 10 machines from the active list, comprising five targets of easy difficulty and five of intermediate difficulty. (2) picoMini [21], a jeopardy-style Capture The Flag (CTF) competition organized by Carnegie Mellon University and redpwn [60]. The competition featured 21 unique CTF chal-

allenges and drew participation from 248 teams in its initial round. These challenges are now freely accessible online for practice and reattempts. Our evaluation employed PENTESTGPT in conjunction with the GPT-4 32k token length API, defining the capture of the root flag as the metric for a successful trial. We conduct five trials on each target and documented the number of successful captures. Note that we consider single successful capture out of five trials as successful attempt over the target. This criterion reflects the iterative nature of real-world penetration testing and CTF challenges, where multiple attempts are allowed, and success is ultimately determined by achieving the objective at least once.

Tables 5 presents PENTESTGPT’s performance across both sets of challenges. In the HackTheBox challenges, PENTESTGPT successfully completed four easy and one medium difficulty challenges, incurring a total cost of 131.5 USD—an average of 21.9 USD per target. This performance indicates PENTESTGPT’s effectiveness in tackling easy to intermediate-level penetration tests at a reasonable cost. Table 6 demonstrates the performance of PENTESTGPT in the picoMini CTF. In particular, PENTESTGPT managed to solve 9 out of 21 challenges, with the average cost per attempt being 5.1 USD. Ultimately, PENTESTGPT accumulated a total of 1400 points⁴ and ranked 24th out of 248 teams with valid submissions [61]. These outcomes suggest a promising performance of PENTESTGPT on real-world penetration testing tasks among various types of challenges.

7 Discussion

It is possible that LLMs used by PENTESTGPT were trained on walkthroughs of the benchmark machines, which could invalidate evaluation results. To counter this, we employ two methods. First, We ensure the LLM lacks prior knowledge of the target machine. We ascertain this by querying LLMs about the tested machine’s familiarity. Secondly, our benchmark comprises machines launched post-2021, ensuring they are beyond OpenAI models’ training data. Our study on recent HackTheBox challenges confirms PENTESTGPT’s ability to solve without pre-existing target knowledge.

While we aim for universally applicable prompts, certain LLMs avoid producing specific hacking content. For instance, OpenAI has implement model alignments [62] to ensure the GPT model outputs do not violate usage policies, including generating malicious exploitation contents. We incorporate jailbreak techniques [63–69] to coax LLMs into producing relevant data. Improving reproducibility of PENTESTGPT remains a focus area.

LLMs occasionally "hallucinate" [57], producing outputs deviating from training data. This impacts our tool’s dependability. To combat this, we’re researching methods [70] to minimize hallucination, anticipating this will boost our tool’s

efficiency and reliability.

The ethical implications of employing PENTESTGPT in penetration testing are significant and warrant careful consideration. While PENTESTGPT can greatly enhance security by identifying vulnerabilities, its capabilities also pose potential risks of misuse. To mitigate these risks, we have implemented several strategies. We actively promote ethical guidelines for the use of PENTESTGPT and collaborate closely with cybersecurity communities to prevent misuse. Moreover, we have incorporated monitoring modules [71] to track the tool’s usage and are committed to ensuring that it is not used inappropriately. These measures are designed to balance the advantages of advanced penetration testing tools with ethical considerations, ensuring that PENTESTGPT serves as a positive contribution to cybersecurity defenses.

8 Conclusion

This work delves into the potential and constraints of LLMs for penetration testing. Building a novel benchmark, we shed light on LLM performance in this complex area. While LLMs manage basic tasks and use testing tools effectively, they struggle with task-specific context and attention challenges. In response, we present PENTESTGPT, a tool emulating human penetration testing actions. Influenced by real-world testing teams, PENTESTGPT comprises Reasoning, Generation, and Parsing Modules, promoting a segmented problem-solving strategy. Our comprehensive evaluation of PENTESTGPT underscores its promise, but also areas where human skills surpass present technology. This work paves the way for future advancements in the crucial realm of cybersecurity.

⁴Each challenge’s points were assigned based on its difficulty level